LEVEL 7 · DIGITAL DESIGN PROJECT

Asynchronous FIFO with Error Monitoring

Project description and complete working explanation

Project details

Developer	Ghulam Yaseen
Date	06 November 2026
Country	Pakistan
HDL standard	Verilog-2001 (IEEE 1364-2001)
Tool	Xilinx Vivado 2025.2.1 Simulator
Target platform	Xilinx FPGA
Design files	`async_fifo.v`, `async_fifo_monitor.v`, `tb_async_fifo_monitor.v`

Contents

This document explains what the Level 7 project does, how every part of the code works, and how the testbench proves it. The sections build on one another, so reading them in order is recommended.

No.	Section	What it covers
1	Project Overview	Purpose, project files, key features and default configuration
2	Background Concepts	Clock domain crossing, Gray code, synchronizers and the wrap bit
3	System Architecture	Block diagram and the main signal paths
4	The Level 6 Core: `async_fifo`	Parameters, ports, write and read operation, full and empty logic, reset
5	The Level 7 Monitor: `async_fifo_monitor`	The two saturating error counters and why they stay in their own clock domains
6	The Testbench	Structure, scoreboard, reference model, helper tasks and the seven tests
7	Worked Example	A step-by-step scenario from reset to a drained FIFO
8	Design Notes	Observations and recommendations on the current code
9	Running the Project in Vivado	Setup steps, expected console output and troubleshooting
10	Summary	Recap and ideas for extending the design

1. Project Overview

This project implements an **asynchronous FIFO** (first-in, first-out buffer) that passes data safely between two circuits running on **unrelated clocks**, and adds a **monitoring layer** that counts every request the FIFO had to refuse. It is the seventh level of a progressive FIFO design series written in Verilog-2001 and developed in Xilinx Vivado.

1.1 Purpose

In real FPGA designs a producer (for example an ADC interface) and a consumer (for example a processing block) often run from different clocks. Copying data directly between them risks lost or corrupted values. An asynchronous FIFO solves this: the producer writes with its own clock, the consumer reads with its own clock, and the FIFO hides the difference between them.

The Level 6 FIFO already ignores a write when it is full and a read when it is empty. Level 7 makes those refusals **visible**: a wrapper counts them, so a designer can tell whether a data link is being overrun (too many writes) or starved (too many reads).

1.2 Project files

File	Module	Role
`async_fifo.v`	`async_fifo`	Level 6 dual-clock FIFO core: memory, Gray-coded pointers, synchronizers and the full and empty flags
`async_fifo_monitor.v`	`async_fifo_monitor`	Level 7 wrapper that instantiates the core and adds two saturating error counters
`tb_async_fifo_monitor.v`	`tb_async_fifo_monitor`	Self-checking testbench for the Level 7 module

1.3 Key features

- Separate **write clock** and **read clock** with no required relationship between them.
- **Gray-coded pointers** passed between domains through **two-flip-flop synchronizers**.
- A **registered read output**: `data_out` is a register that holds the last word read.
- **Conservative flags**: `full` and `empty` may react a little late, but never report space or data that is not there.
- Two **saturating error counters**, each living in the clock domain of the request it watches.
- An independent **active-low asynchronous reset** for each clock domain.
- A **self-checking testbench** with a data scoreboard, a reference model of the counters, and random traffic.

1.4 Default configuration

Parameter	Value	Meaning
`DATA_WIDTH`	8	Each stored word is 8 bits wide
`ADDR_WIDTH`	3	Three address bits select a memory location
`DEPTH`	8	Eight words can be stored (2 to the power 3)
Pointer width	4	`ADDR_WIDTH` + 1 bits; the extra bit is the wrap bit
`ERROR_WIDTH`	8	Each error counter is 8 bits wide and saturates at 255

2. Background Concepts

Four ideas explain why the design looks the way it does. Each one is a standard technique for moving data between clock domains.

2.1 What a FIFO does

A FIFO is a buffer in which words leave in the same order they entered. A **write pointer** says where the next word will be stored, and a **read pointer** says where the next word will be taken from. When the two pointers meet, the FIFO is either empty or full, depending on which pointer caught up with the other.

2.2 Why crossing clock domains is difficult

A flip-flop that samples a signal changing close to its clock edge can enter an unstable state called **metastability**, where its output is neither a clean 0 nor a clean 1 for a short time.

The write side needs to know how far the reader has got, and the read side needs to know how far the writer has got. Sending a plain binary counter across is unsafe, because several bits can change at once (for example 0111 becomes 1000, a change in four bits). The receiving clock may sample some bits before the change and some after, producing a value that was never valid.

2.3 Gray code

In a Gray code, **only one bit changes** when the value goes up by one. A receiving clock that samples during the change sees either the old value or the new value, and both are valid pointer positions. The conversion used in the design is:

```
gray = (binary >> 1) ^ binary;
```

Dec	Binary	Gray	Dec	Binary	Gray
0	0000	0000	8	1000	1100
1	0001	0001	9	1001	1101
2	0010	0011	10	1010	1111
3	0011	0010	11	1011	1110
4	0100	0110	12	1100	1010
5	0101	0111	13	1101	1011
6	0110	0101	14	1110	1001
7	0111	0100	15	1111	1000

Table 1. Four-bit binary to Gray code conversion (the pointer width used by default).

2.4 Two-flip-flop synchronizer

A Gray-coded pointer from the other domain passes through **two flip-flops** clocked by the local clock. The first flip-flop may go metastable, but it has a full clock period to settle before the second one samples it. The second flip-flop therefore delivers a clean value. The price is a delay of about two local clock cycles.

2.5 The wrap bit

Each pointer is one bit wider than the memory address. The lower bits choose the memory location, while the top bit toggles every time the pointer passes the end of the memory. This extra bit separates the two cases in which the pointers point to the same location:

- **Empty**: the two pointers are completely equal, including the wrap bit.
- **Full**: the addresses match but the wrap bits differ, meaning the writer is exactly one lap ahead of the reader.

3. System Architecture

Figure 1 shows the complete design. The outer box is the Level 7 module `async_fifo_monitor`. Inside it sits the Level 6 core `async_fifo`, split into a write domain (blue) and a read domain (green), with the shared memory between them. The two yellow boxes are the synchronizers that carry Gray-coded pointers across. The orange boxes are the Level 7 error counters.

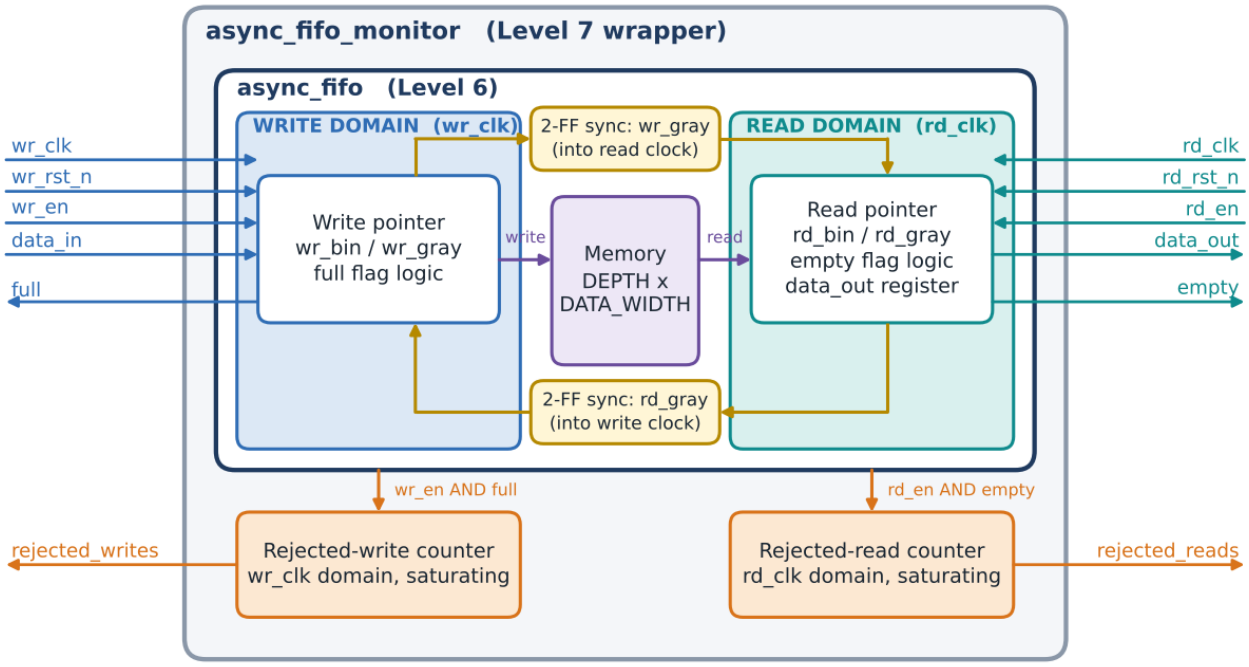


Figure 1. Architecture of the Level 7 asynchronous FIFO with error monitoring.

3.1 Main signal paths

Path	From to	What happens
Write data path	`data_in` to memory	The word is stored at the location addressed by the lower bits of `wr_bin` when a write is accepted
Read data path	memory to `data_out`	The word at the read address is copied into the output register when a read is accepted
Write pointer crossing	`wr_gray` through two flops in `rd_clk` to the `empty` logic	Tells the read side how much data has been written
Read pointer crossing	`rd_gray` through two flops in `wr_clk` to the `full` logic	Tells the write side how much space has been freed
Error monitoring	`wr_en` AND `full`, and `rd_en` AND `empty`, to the counters	Each counter stays in the clock domain of the request it watches

4. The Level 6 Core: async_fifo

The core is the heart of the project. Each clock domain owns its own binary pointer, and only the Gray-coded form of that pointer is sent to the other domain.

4.1 Parameters

Parameter	Default	Description
`DATA_WIDTH`	8	Width of each stored word in bits
`ADDR_WIDTH`	3	Number of address bits; the depth is 2 to the power `ADDR_WIDTH`. Must be at least 2
`DEPTH`	1 << `ADDR_WIDTH`	Number of memory locations. Must equal 2 to the power `ADDR_WIDTH`
`PTR_WIDTH` (local)	`ADDR_WIDTH` + 1	Pointer width including the wrap bit

4.2 Ports

Port	Direction	Width	Domain	Description
wr_clk	Input	1	Write	Write clock; all write-side registers update on its rising edge
rd_clk	Input	1	Read	Read clock; all read-side registers update on its rising edge
wr_rst_n	Input	1	Write	Active-low asynchronous reset for the write domain
rd_rst_n	Input	1	Read	Active-low asynchronous reset for the read domain
wr_en	Input	1	Write	Write request; accepted only when full is low
data_in	Input	DATA_WIDTH	Write	Word to be stored
rd_en	Input	1	Read	Read request; accepted only when empty is low
data_out	Output (reg)	DATA_WIDTH	Read	Registered read data; holds the last word read
full	Output	1	Write	High when no free location is left, as seen by the writer
empty	Output	1	Read	High when no unread word is left, as seen by the reader

4.3 Internal signals

Signal	Width	Domain	Purpose
memory	DATA_WIDTH x DEPTH	Both	Storage array, written by wr_clk and read by rd_clk
wr_bin	PTR_WIDTH	Write	Binary write pointer; lower bits address the memory, top bit is the wrap bit
wr_gray	PTR_WIDTH	Write	Gray-coded copy of wr_bin; this is the value sent to the read domain
rd_bin	PTR_WIDTH	Read	Binary read pointer
rd_gray	PTR_WIDTH	Read	Gray-coded copy of rd_bin; this is the value sent to the write domain
rd_gray_w1, rd_gray_w2	PTR_WIDTH	Write	Two-flop synchronizer that brings rd_gray into wr_clk
wr_gray_r1, wr_gray_r2	PTR_WIDTH	Read	Two-flop synchronizer that brings wr_gray into rd_clk
wr_bin_next, rd_bin_next	PTR_WIDTH	Write / Read	Pointer value after this clock edge: the pointer plus one only if the request is accepted
wr_gray_next, rd_gray_next	PTR_WIDTH	Write / Read	Gray conversion of the next pointer value

4.4 Write operation

Everything on the write side happens on the rising edge of `wr_clk`:

1. A write is **accepted** when `wr_en` is high and `full` is low.
2. `data_in` is stored in `memory` at the address given by the lower `ADDR_WIDTH` bits of `wr_bin`.
3. `wr_bin` moves to `wr_bin_next` (one higher) and `wr_gray` is updated to the matching Gray code.
4. If `full` is high, the request is ignored: the memory and both write pointers stay unchanged.

4.5 Read operation

Everything on the read side happens on the rising edge of `rd_clk`:

1. A read is **accepted** when `rd_en` is high and `empty` is low.
2. The word at the address given by the lower bits of `rd_bin` is copied into the `data_out` register.
3. `rd_bin` and `rd_gray` advance by one.
4. If `empty` is high, the request is ignored and `data_out` keeps its previous value.

> **Read latency**
> `data_out` is a register, so the word requested by an accepted read appears on `data_out` just after that clock edge, and can be used in the following read-clock cycle. It then stays there until the next accepted read.

4.6 Pointer synchronization

Two small always blocks carry the pointers across. In the write domain, `rd_gray` is sampled into `rd_gray_w1` and then `rd_gray_w2`. In the read domain, `wr_gray` is sampled into `wr_gray_r1` and then `wr_gray_r2`. Only the second flop of each pair is used by the flag logic. Both pairs are cleared to zero by the reset of their own domain.

4.7 The empty flag

```
assign empty = (rd_gray == wr_gray_r2);
```

The read side compares its own pointer with a delayed copy of the write pointer. If they are equal, every word that has been written has also been read, so the FIFO is empty.

4.8 The full flag

```
assign full = (wr_gray == {~rd_gray_w2[PTR_WIDTH-1:PTR_WIDTH-2], rd_gray_w2[PTR_WIDTH-3:0]});
```

A write pointer that is exactly one full lap ahead of the read pointer has a Gray code that differs from the read pointer's Gray code in its **top two bits** and matches in all the others. The logic therefore inverts the top two bits of the synchronized read pointer and compares the result with `wr_gray`. This is also why `ADDR_WIDTH` must be at least 2.

Worked example (DEPTH = 8, pointer width 4)

rd_bin	wr_bin	wr_gray	Inverted top bits of rd_gray	Equal?	Result
0 (0000)	8 (1000)	1100	1100	Yes	**Full**: eight words stored
1 (0001)	8 (1000)	1100	1101	No	Not full: seven words stored
8 (1000)	8 (1000)	1100	0100	No	Not full (this is the empty case)

4.9 Flag latency and safety

Each domain sees the other domain's pointer about two clock cycles late. This delay only errs in the **safe direction**:

- The write side sees an old read pointer, so the FIFO may look **fuller** than it is. `full` can stay high for a couple of write clocks after a read has freed a location.
- The read side sees an old write pointer, so the FIFO may look **emptier** than it is. `empty` can stay high for a couple of read clocks after a write has stored a word.
- Neither side can ever overwrite unread data or read data that has not been stored, because pointers only move forward.

4.10 Reset behavior

Each domain has its own active-low asynchronous reset. `wr_rst_n` clears `wr_bin`, `wr_gray` and the synchronizer that brings the read pointer into the write domain. `rd_rst_n` clears `rd_bin`, `rd_gray`, `data_out` and the synchronizer that brings the write pointer into the read domain. The memory contents are not reset; they become irrelevant because both pointers return to zero.

5. The Level 7 Monitor: async_fifo_monitor

The monitor is a thin wrapper. It passes every signal straight through to `async_fifo` and watches the request and flag signals to count refused requests. The FIFO's behavior is unchanged.

5.1 Parameters

Parameter	Default	Description
`DATA_WIDTH`	8	Width of each FIFO word; passed to the core
`ADDR_WIDTH`	3	Address width of the FIFO; passed to the core
`DEPTH`	1 << `ADDR_WIDTH`	Number of FIFO entries; passed to the core
`ERROR_WIDTH`	8	Width of each error counter in bits; the largest value is 2 to the power `ERROR_WIDTH` minus 1 (255 by default)

5.2 Ports

All FIFO ports are passed through unchanged. The monitor adds the two counter outputs at the bottom of the table.

Port	Direction	Width	Description
wr_clk, rd_clk	Input	1 each	Write and read clocks
wr_rst_n, rd_rst_n	Input	1 each	Active-low resets for the write and read domains
wr_en, data_in	Input	1, DATA_WIDTH	Write request and write data
rd_en	Input	1	Read request
data_out	Output	DATA_WIDTH	Read data from the core
full, empty	Output	1 each	Status flags from the core
rejected_writes	Output (reg)	ERROR_WIDTH	Number of writes attempted while the FIFO was full
rejected_reads	Output (reg)	ERROR_WIDTH	Number of reads attempted while the FIFO was empty

5.3 Module structure

- **Section 1** instantiates the core as u_fifo, passing all parameters and connecting every port.
- **Section 2** is the rejected-write counter, clocked by wr_clk.
- **Section 3** is the rejected-read counter, clocked by rd_clk.

5.4 How the counters work

Counter	Clock	Increments when	Stops at	Cleared by
rejected_writes	wr_clk	wr_en is high while full is high	All ones (255 by default)	wr_rst_n low
rejected_reads	rd_clk	rd_en is high while empty is high	All ones (255 by default)	rd_rst_n low

```
always @(posedge wr_clk or negedge wr_rst_n) begin
    if (!wr_rst_n) begin
        rejected_writes <= {ERROR_WIDTH{1'b0}};
    end
    else begin
        if (wr_en && full && (rejected_writes != {ERROR_WIDTH{1'b1}})) begin
            rejected_writes <= rejected_writes + 1'b1;
        end
    end
end
```

The read counter has exactly the same form, using rd_clk, rd_en and empty. Three properties are worth noting:

- **Saturation.** The test against all ones stops the counter at its maximum value. A counter that wrapped from 255 back to 0 could hide a serious overflow; a saturated counter always shows that the limit was reached.
- **No clock domain crossing.** wr_en and full both belong to the write domain, and rd_en and empty both belong to the read domain. Each counter therefore sits in the same domain as the signals it watches and needs no synchronizer.

- **Exact agreement with the core.** A write is either accepted (`wr_en` and not `full`) or rejected (`wr_en` and `full`). The counter uses the very same `full` signal the core uses, so a request counts as rejected exactly when the core ignored it. The same holds for reads and `empty`.

6. The Testbench

`tb_async_fifo_monitor` is a **self-checking** testbench: it applies stimulus, compares the results with independent expectations, and prints a pass or fail summary without needing anyone to inspect waveforms.

6.1 Clocks

The write clock has a 10 ns period (100 MHz) and the read clock a 14 ns period (about 71.4 MHz). Because the two periods share no simple ratio, the phase between the clocks keeps changing, which exercises the synchronizers far better than two related clocks would.

6.2 Parameters

Parameter	Default	Meaning
`DATA_WIDTH`	8	Word width of the DUT
`ADDR_WIDTH`	3	Address width of the DUT
`DEPTH`	1 << `ADDR_WIDTH`	FIFO depth (8)
`ERROR_WIDTH`	8	Width of the error counters
`WR_PERIOD`	10	Write clock period in ns
`RD_PERIOD`	14	Read clock period in ns
`READ_REGISTERED`	1	1 if `data_out` is registered (as in the Level 6 core); 0 if it shows the word in the same cycle the read is accepted
`SCB_SIZE`	4096	Number of entries in the scoreboard memory
`RANDOM_CYCLES`	1500	Write-clock cycles in each random traffic run
`ERR_MAX` (local)	All ones	Largest counter value (255 by default)

> **Keep READ_REGISTERED at 1**
> The Level 6 core declares `data_out` as a register, so the testbench must use `READ_REGISTERED = 1`. With 0, every data comparison would look at the output one cycle too early and report false mismatches.

6.3 Structure of the testbench

Section	Block	Purpose
1–2	Parameters and signals	Declares the settings, the DUT connections, the scoreboard and the result counters
3	Device under test	Instantiates `async_fifo_monitor` as `dut`
4	Clock generation	Two free-running clocks with the periods above
5	Reference model	An independent copy of the counter rules, used to predict `rejected_writes` and `rejected_reads`
6	Scoreboard	Records every accepted write and checks every accepted read against it
7	Continuous checkers	Compare the DUT counters with the model every clock cycle and check that `full` and `empty` are never high together
8	Random traffic drivers	Generate random write and read requests with chosen probabilities
9	Helper tasks	Reusable checking and stimulus routines (section 6.6)
10	Main test sequence	Runs the seven tests and prints the summary
11	Watchdog timer	Ends the simulation with a failure message if it runs for 20 ms

6.4 The scoreboard

On every `wr_clk` edge where `wr_en` is high and `full` is low, the scoreboard stores `data_in` in its own memory and moves its write index forward. On every `rd_clk` edge where `rd_en` is high and `empty` is low, the read index moves forward and the matching stored word becomes the **expected value**.

Because `data_out` is registered, the comparison is made on the **next** read-clock edge, when the new word is guaranteed to be on `data_out`. Any difference prints a data mismatch message and increments the error count. This single mechanism checks that no word is lost, repeated, corrupted or delivered out of order.

6.5 The reference model and continuous checkers

The testbench contains its own copy of the counter rules (increment on a refused request, stop at all ones, clear on reset). On every falling clock edge the DUT counters are compared with this model, so a counter error is caught the moment it appears rather than only at the end of a test. A third checker reports an error if `full` and `empty` are ever high at the same time.

6.6 Helper tasks

Task	Description
`check_cond`	Counts a check and reports an error with a label if a condition is false
`check_value`	Counts a check and reports an error if a value differs from the expected value
`wr_burst(n)`	Holds `wr_en` high with random data for `n` write-clock cycles, then lowers it
`rd_burst(n)`	Holds `rd_en` high for `n` read-clock cycles, then lowers it
`settle`	Waits six cycles of each clock so pointer synchronizers and flags can settle
`apply_reset`	Lowers both resets on falling clock edges, holds them for four cycles of each clock, releases them on falling edges, then settles
`random_run(w, r, n)`	Runs random traffic for `n` write-clock cycles with write probability `w` percent and read probability `r` percent
`drain_fifo`	Reads until the FIFO reports empty

Stimulus is always changed with non-blocking assignments just after a clock edge, and resets are released on falling edges. This avoids race conditions between the testbench and the DUT.

6.7 Test plan

Test	Stimulus	What is checked
1	Apply reset	`empty` is high, `full` is low, both counters are 0
2	Five reads on the empty FIFO	`rejected_reads` becomes 5 and `rejected_writes` stays 0
3	Write `DEPTH` words, then four more writes	`full` goes high after the eighth word, `empty` goes low, and `rejected_writes` becomes 4
4	Read `DEPTH` words, then three more reads	Data arrives in order (scoreboard), `empty` goes high, `full` goes low, and `rejected_reads` becomes 8 while `rejected_writes` stays 4
5	Fill the FIFO, then `ERR_MAX` + 10 (265) extra writes; drain, then 265 extra reads	Both counters saturate at 255 instead of wrapping around
6	Apply reset again	Both counters return to 0, `empty` is high, `full` is low
7	Four random traffic runs (write / read probabilities 50/50, 90/20, 20/90 and 70/70 percent, 1500 write-clock cycles each), then drain	Scoreboard data checks and counter model checks throughout; the FIFO ends empty and the scoreboard read and write counts match

6.8 Reporting

At the end the testbench prints the number of checks performed and the number of errors found, followed by `RESULT: PASSED` if there were no errors or `RESULT: FAILED` otherwise, and then calls `$finish`. If the simulation ever runs for 20 ms, the watchdog prints a failure message and ends it, so a stuck test can never hang the simulator.

7. Worked Example

The table below follows one realistic scenario with the default configuration (DEPTH = 8). The delays of "about two clocks" come from the synchronizers described in section 4.9.

Step	Event	Result
1	Both resets are released	All pointers are 0. empty = 1, full = 0, both counters are 0.
2	The reader asserts rd_en while the FIFO is empty	The request is ignored. rejected_reads becomes 1.
3	The writer writes eight words	wr_bin goes from 0 to 8 (binary 1000) and wr_gray becomes 1100. The read pointer is still 0, so full goes high.
4	The writer tries a ninth write	The request is ignored and no stored word is overwritten. rejected_writes becomes 1.
5	About two read clocks after the first write	The new wr_gray reaches the read domain and empty goes low.
6	The reader reads one word	rd_bin goes from 0 to 1. The oldest word appears on data_out just after the clock edge.
7	About two write clocks later	The new rd_gray reaches the write domain and full goes low, so the writer can write again.
8	The reader reads the remaining words	empty goes high once rd_gray equals the synchronized wr_gray. The counters keep their values until the next reset.

8. Design Notes

These observations come from reading the three files. None of them stops the design from working as described; they are points to be aware of when reusing or extending it.

Comment on the full flag

The comment above the full assignment in async_fifo.v says it compares the "next" write Gray pointer, but the code compares the current wr_gray. The logic is correct for this design, because full is recomputed from the registers after every accepted write. Only the comment needs to be updated.

Parameter limits

DEPTH must equal 2 to the power ADDR_WIDTH, and ADDR_WIDTH must be at least 2. The full-flag expression uses PTR_WIDTH-3 as a bit index, which would be invalid for smaller values.

Reset both domains together

Each domain has its own reset, but the two pointers describe one shared FIFO. Resetting only one domain would leave the pointers inconsistent. The testbench always applies and releases both resets together. In a real FPGA design, each reset release should also be synchronized to its own clock (a reset synchronizer), which the educational Level 6 core does not include.

> **Counters belong to their own domain**
> `rejected_writes` is stable in the write domain and `rejected_reads` in the read domain. If logic in a third clock domain needs to read either value, add a proper synchronization method for multi-bit values (for example Gray coding or a handshake) at that point.

9. Running the Project in Vivado

9.1 Setup steps

1. Add `async_fifo.v` and `async_fifo_monitor.v` as design sources.
2. Add `tb_async_fifo_monitor.v` as a simulation source and set `tb_async_fifo_monitor` as the simulation top module.
3. Make sure all three files are in the project and enabled. The monitor instantiates `async_fifo`, so elaboration fails if that file is missing.
4. Run Behavioral Simulation and watch the Tcl console for the test messages and the summary.

9.2 Expected console output

A clean run ends with a summary of this form (the number of checks will vary):

```
================================================
 Checks performed : <number>
 Errors found     : 0
 RESULT           : PASSED
================================================
```

9.3 Troubleshooting

Symptom	Likely cause	Fix
Elaboration fails with module `async_fifo` not found	`async_fifo.v` is not in the project sources	Add the Level 6 file to the project and enable it
Many “data mismatch” messages	`READ_REGISTERED` is set to 0 for a registered `data_out`	Set `READ_REGISTERED` to 1 in the testbench
Simulation ends with a watchdog message	Wrong simulation top, or the sequence is stuck waiting	Set `tb_async_fifo_monitor` as the simulation top and re-run
Counter error messages	A real mismatch between the DUT counters and the model	Inspect `wr_en`, `full`, `rd_en`, `empty` and the counters around the printed time

10. Summary

The Level 7 design combines three pieces. The Level 6 core moves data safely between two unrelated clocks using Gray-coded pointers, two-flip-flop synchronizers and an extra wrap bit to tell full from empty. The Level 7 wrapper adds two saturating counters that record every refused write and read without crossing any clock domain. The testbench checks the whole design with a data scoreboard, an independent counter model, directed corner cases and random traffic.

10.1 Ideas for extending the design

- Add **almost-full** and **almost-empty** flags to give the producer and consumer early warning.
- Add a **fill-level** output derived from the synchronized pointers.
- Pass the error counters to a single status clock domain using a safe multi-bit synchronization method.
- Add reset synchronizers for each domain.
- Add assertions or formal properties for the pointer and flag rules.